Εργαστήριο HLS: Επιτάχυνση Υλικού Αλγορίθμου Row Contrast Stretch (Vitis HLS)

Όνομα Φοιτητή: Πελοπίδας-Νικόλαος Τσιούντσιουρας
A.E.M.: 11085

13 Νοεμβρίου 2026

1 Εισαγωγή και Περιγραφή Υλοποίησης

Στο πλαίσιο της παρούσας εργασίας, αναπτύχθηκε και βελτιστοποιήθηκε ένας επιταχυντής υλικού για τον αλγόριθμο **Row Contrast Stretch** χρησιμοποιώντας το περιβάλλον **Xilinx Vitis HLS 2022.2**. Ο αλγόριθμος επεξεργάζεται μια εικόνα/πίνακα διαστάσεων $HEIGHT \times WIDTH$ (όπου $HEIGHT = WIDTH = 256$ στην αρχική διαμόρφωση) σειριακά ανά γραμμή. Υπολογίζει τις ελάχιστες και μέγιστες τιμές (`row_min`, `row_max`) κάθε γραμμής, παράγει συντελεστή κλιμάκωσης και επανεγγραφεί τα pixel κανονικοποιημένα στο εύρος $[0, 255]$.

Η διαδικασία επαλήθευσης περιλάμβανε τρία βασικά στάδια:

- **C Simulation (CSIM):** Έλεγχος ορθότητας του αλγορίθμου σε επίπεδο λογισμικού με χρήση testbench.
- **C Synthesis (CSYNTH):** Μετάφραση του κώδικα C σε περιγραφή υλικού RTL, εξαγωγή εκτιμήσεων χρονισμού, λανθάνων χρόνου (latency) και χρήσης πόρων (LUT, FF, DSP, BRAM).
- **C/RTL Cosimulation (COSIM):** Έλεγχος ταυτόχρονης λειτουργίας λογισμικού και παραγόμενου υλικού για την επιβεβαίωση της ακρίβειας bit-to-bit (PASS) και μέτρηση του συνολικού χρόνου εκτέλεσης.

2 Βασική Υλοποίηση (Baseline - Ερωτήματα 2 & 3)

Στην αρχική unoptimized έκδοση, ο κώδικας εκτελούσε τους υπολογισμούς σειριακά χωρίς οδηγίες βελτιστοποίησης (pragmas). Τα αποτελέσματα σύνθεσης και συνσομοίωσης για διαστάσεις 256×256 αποτυπώνονται στον Πίνακα 1.

3 Βελτιστοποίηση Υλικού (Ερώτημα 4)

Για τη μείωση του λανθάνων χρόνου χωρίς υπέρβαση των χρονικών ορίων, εφαρμόστηκαν οδηγίες (**Pragmas**) στοχεύοντας στην εξάλειψη των περιορισμών μνήμης και στη δυνατότητα παράλληλης εκτέλεσης:

Table 1: Μετρικές Βασικής Υλοποίησης (Baseline) για $HEIGHT = WIDTH = 256$

Μετρική Σύνθεσης / Συνσομοίωσης	**Τιμή (Baseline)**
Εκτιμώμενη Περίοδος Ρολογιού (Estimated Clock Period)	3.199 ns (Στόχος: 10.00 ns)
Χειρότερη Λανθάνουσα Κατάσταση (Worst-case Latency)	1.411 μs (141,057 κύκλοι)
Χρήση DSP48E	1
Χρήση BRAMs	0
Χρήση Flip-Flops (FFs)	560
Χρήση Look-Up Tables (LUTs)	870
Συνολικός Χρόνος Εκτέλεσης (Total Execution Time)	60.597 s
Ελάχιστη / Μέση / Μέγιστη Λανθάνουσα (Cosim Latency)	140,003 / 140,003 / 140,003 κύκλοι

1. **Array Partitioning:** Χρησιμοποιήθηκε κυκλικός καταμερισμός (`type=cyclic factor=8 dim=2`) στον πίνακα εισόδου `A`, επιτρέποντας την ταυτόχρονη ανάγνωση πολλαπλών δεδομένων ανά κύκλο ρολογιού χωρίς να δημιουργούνται bottlenecks στις θύρες μνήμης της BRAM/URAM.
2. **Loop Pipelining:** Εφαρμόστηκε `#pragma HLS PIPELINE II=1` στους ανεξάρτητους βρόχους κλιμάκωσης και εγγραφής του πίνακα `C`, ώστε να εκκινεί νέα επανάληψη σε κάθε κύκλο ρολογιού (Initiation Interval $II = 1$).

3.1 Κλιμάκωση Διαστάσεων Πλάτους (Ερώτημα 4i)

Διατηρώντας σταθερό το ύψος ($HEIGHT = 256$) και μεταβάλλοντας το πλάτος ($WIDTH \in \{64, 128, 256, 512\}$), συλλέχθηκαν οι μετρικές του Πίνακα 2.

Table 2: Μετρικές Βελτιστοποιημένου Συστήματος ανά Μεταβολή Πλάτους (Width Scaling)

Μετρική	**64**	**128**	**256**	**512**
Εκτιμώμενη Περίοδος Ρολογιού	3.458 ns	3.515 ns	3.533 ns	3.533 ns
Χειρότερη Λανθάνουσα (ns)	4.280×10^5	7.550×10^5	1.411×10^6	2.721×10^6
Χρήση DSP48E	1	1	1	1
Χρήση BRAMs	0	0	0	0
Χρήση FFs	550	555	560	565
Χρήση LUTs	1,246	1,251	1,256	1,261
Συνολικός Χρόνος Εκτέλεσης (s)	62.005 s	44.461 s	41.284 s	53.765 s
Λανθάνουσα (Κύκλοι - Cycles)	27,875	74,467	14,003	271,065

3.2 Βέλτιστη Διαμόρφωση και Υπολογισμός Speed-Up (Ερώτηματα 4ii & 4iii)

Για την τελική βέλτιστη διαμόρφωση με $HEIGHT = WIDTH = 256$, ο συνδυασμός του κυκλικού καταμερισμού μνήμης και του pipelining μείωσε τους κύκλους από $141,057$ σε $14,003$.

Ο συντελεστής επιτάχυνσης (**Speed-Up**) υπολογίζεται βάσει της μείωσης των κύκλων ρολογιού: Speed-Up $= \frac{\text{Baseline Cycles}}{\text{Optimized Cycles}} = \frac{141,057}{14,003} \approx \mathbf{10.07}\times$

Table 3: Σύγκριση Baseline και Βέλτιστης Βελτιστοποιημένης Έκδοσης (256×256)

Χαρακτηριστικό	**Baseline**	**Βέλτιστο (Optimized)**	**Μεταβολή**
Κύκλοι Λανθάνουσας (Latency Cycles)	141,057	14,003	-90.07%
Χρήση LUTs	870	1,256	+44.3%
Χρήση FFs	560	560	0%
Εκτιμώμενη Περίοδος Ρολογιού	3.199 ns	3.533 ns	+10.4%

3.3 Ανάλυση Εναλλακτικής Υπερ-Βελτιστοποίησης (Over-Optimization Trade-Off)

Κατά τη διερεύνηση επιπλέον βελτιστοποιήσεων, επιχειρήθηκε η εισαγωγή επιθετικού unrolling (`#pragma HLS UNROLL factor=16`) στον εσωτερικό βρόχο εύρεσης ελαχίστων/μεγίστων. Παρότι θεωρητικά αυξάνει τον παραλληλισμό, τα αποτελέσματα έδειξαν σημαντική επιδείνωση:

- Η εκτιμώμενη περίοδος ρολογιού εκτινάχθηκε από τα 3.53 ns στα 7.07 **ns**, λόγω της επιβάρυνσης του κρίσιμου μονοπατιού (critical path) από τα πολυπληθή πολυπλεγμένα δέντρα συγκρίσεων.
- Η χρήση LUTs τριπλασιάστηκε ξεπερνώντας τα **3,015 LUTs**.

Το γεγονός αυτό αποδεικνύει ότι η τυφλή εφαρμογή επιθετικού unrolling σε πράξεις εξάρτησης μείωσης (reduction dependencies) καταστρέφει τα χρονικά περιθώρια (timing slack) χωρίς αναλογικό κέρδος επιδόσεων, δικαιώνοντας την επιλογή του μέτριου pipelining και partition ως την πιο ορθολογική μηχανική λύση.

4 Συμπεράσματα

Η υλοποίηση και επιτάχυνση του αλγορίθμου *Row Contrast Stretch* ανέδειξε την κρίσιμη σημασία της σωστής διαχείρισης μνήμης (Array Partitioning) και του pipelining στο σχεδιασμό επιταχυντών υλικού. Επιτεύχθηκε εντυπωσιακή επιτάχυνση **10.07**$\times$ σε σχέση με τη βασική σειριακή δομή, διατηρώντας απόλυτη συμβατότητα χρονισμού και χαμηλή κατανάλωση πόρων.